IA para Desarrolladores

Sesión 1 · Entender: cómo "lee" un LLM tu código

CURSO · 20 HORAS · 10 SESIONES

Agenda de hoy

10'

Apertura, acuerdos y reglas de datos

20'

Qué hay dentro de un LLM

45'

Las herramientas: qué son, cómo se usan y sus modelos

5'

Demo: la IA se equivoca con seguridad

25'

Manos a la obra: un simulador de crédito

15'

Límites, reglas y cómo llegar a la sesión 2

Hoy Copilot y Claude Code corren en mi pantalla; desde la sesión 2, en la suya.

20 horas, 5 módulos, un reto que crece

1

Entender

Fundamentos de IA

S1

2

Explorar

Modelos, APIs y LLMs

S2 · Hito 1

3

Hacer

Desarrollo asistido por IA

S3–S4 · Hito 2

4

Transformar

Programación asistida, QA y seguridad

S5–S8 · Hitos 3 y 4

5

Apropiar

Reto aplicado a su trabajo

S9–S10 · Demo final

10 sesiones de 2 horas, 2 por semana. Cada una: concepto → demo en vivo → laboratorio → puesta en común.

Reto en equipos de 4–5: se define en la sesión 2 y avanza por hitos hasta la demo de la sesión 10.

Certificación: asistencia mínima del 80 % (máximo 2 inasistencias).

Regla número uno: qué puede ver la IA

Sí, en las herramientas autorizadas

- Código y datos sintéticos de los ejercicios del curso
- Herramientas del curso: GitHub Copilot y Claude Code, en VS Code y en terminal
- Código interno, solo según la política de la organización
- Preguntas conceptuales y de documentación

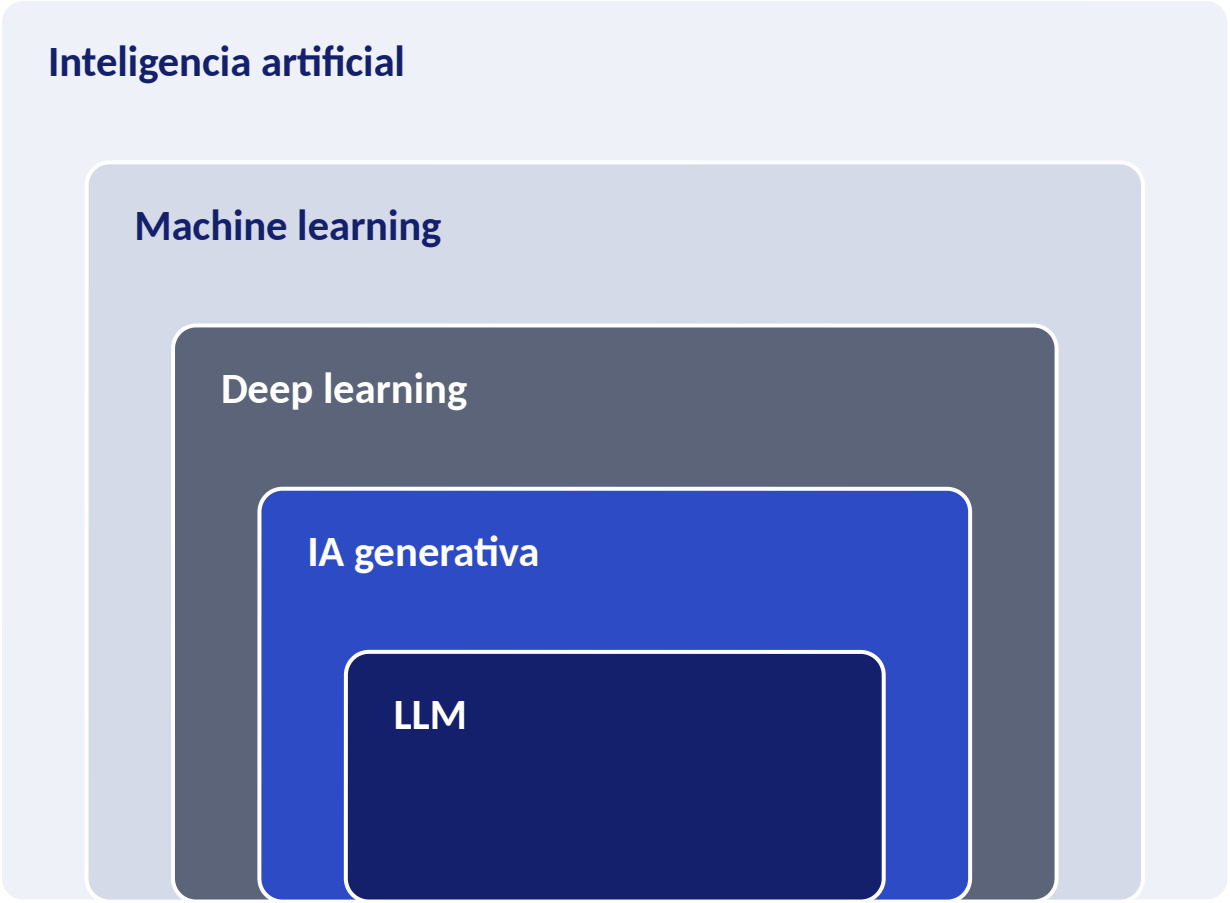
Nunca, en ninguna herramienta

- Datos de clientes: nombres, cédulas, cuentas, saldos
- Credenciales, llaves, tokens, cadenas de conexión
- Información clasificada como confidencial
- Cualquier cosa en herramientas no aprobadas (cuentas personales, extensiones sin revisar)

¿Qué hay dentro de un LLM?

Lo mínimo que un desarrollador necesita saber para usarlo bien

De la IA al LLM: cada término está dentro del anterior



Inteligencia artificial: Sistemas que realizan tareas que asociamos a la inteligencia

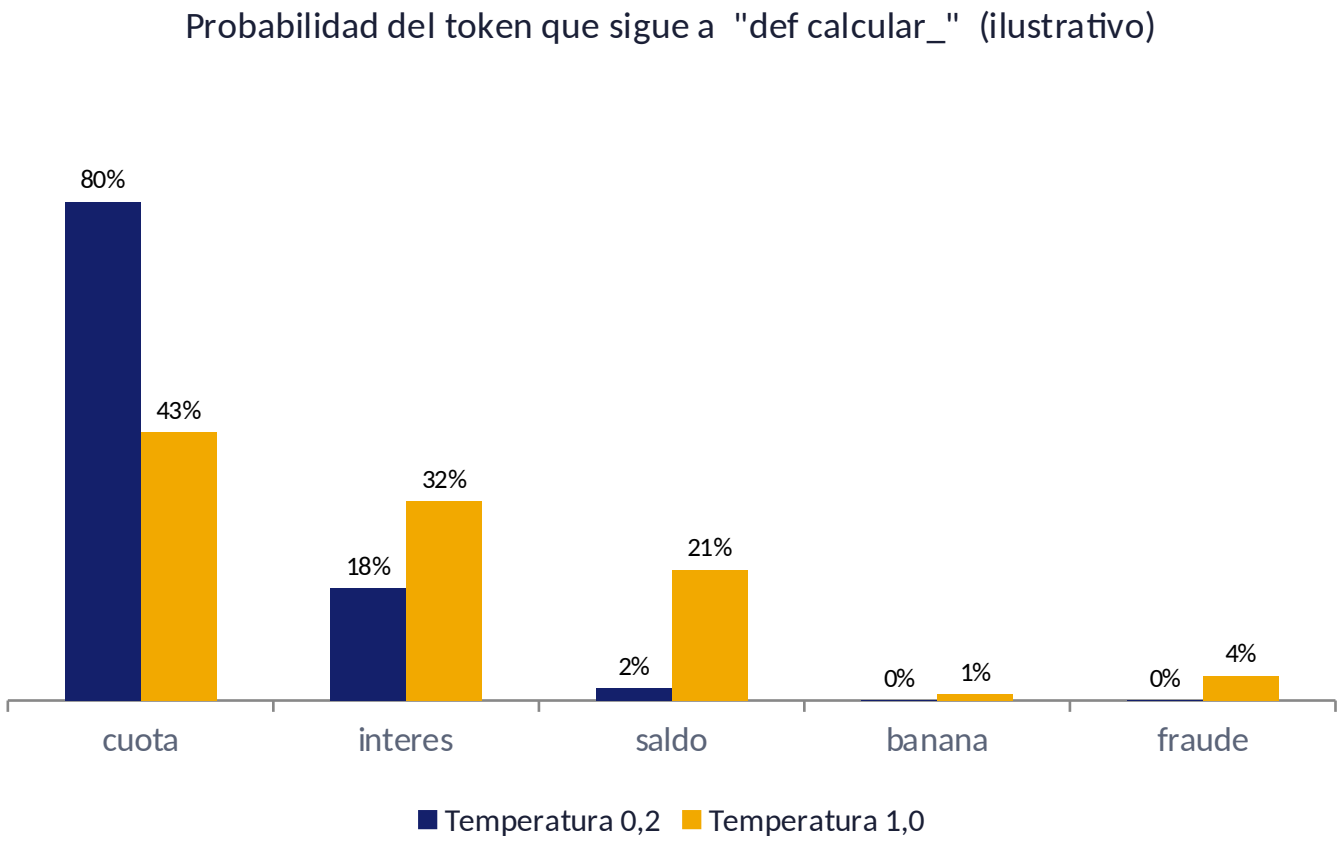
Machine learning: Aprende patrones de datos en lugar de reglas escritas

Deep learning: Redes neuronales con muchas capas

IA generativa: Produce contenido nuevo: texto, código, imágenes

LLM: Modelo de lenguaje entrenado con billones de tokens de texto y código

Un LLM hace una sola cosa: predecir el siguiente token



Por eso…

- La misma pregunta puede dar respuestas distintas.
- El modelo siempre responde algo, aunque no sepa.
- Una respuesta fluida no es una respuesta correcta.

Tokens: la unidad con la que se mide todo

El modelo no lee caracteres ni palabras: lee fragmentos llamados tokens.



Se cobra por token

Las APIs facturan tokens de entrada y de salida. Más contexto = más costo.

Hay un límite

La ventana de contexto es finita: lo que no cabe, el modelo no lo ve.

La forma importa

Español > inglés, JSON > YAML, identificadores largos se parten en varios.

El recorrido dentro del transformer

01

Texto

def calcular_

›

02

Tokens

[755, 11294, 62]

›

03

Embeddings

un vector por token

›

04

N capas

atención + red densa

›

05

Probabilidad

sobre todo el vocabulario

›

06

Siguiente token

"interes"

Se repite en bucle. El token generado se agrega a la entrada y el recorrido vuelve a empezar. Una respuesta de 500 tokens son 500 vueltas: por eso la salida cuesta más y tarda más que la entrada.

Atención: cada token decide a quién mirar

	saldo	=	cuenta	.	saldo	-	monto
saldo	1,00						
=	0,02	0,98					
cuenta	0,14	·	0,85				
.	0,01	0,57	0,02	0,40			
saldo	0,48	·	0,04	·	0,48		
-	0,01	0,37	0,01	0,24	0,01	0,37	
monto	**0,32**	·	0,02	·	**0,32**	·	**0,35**

Fila = token que pregunta · columna = token observado · gris = futuro (máscara causal)

$$\text{softmax}(Q \cdot K^T / \sqrt{d}) \cdot V$$

'monto' mira a 'saldo' y casi nada a los operadores.

Solo mira hacia atrás: por eso un modelo generativo predice el siguiente token.

Consecuencia práctica: si la regla de negocio no está en el contexto, no hay a qué prestarle atención.

Embeddings: buscar código por significado

Cada fragmento se convierte en un vector; "parecido" = similitud coseno alta. Así encuentra un asistente el código relevante para darle contexto al modelo.

"ocultar número de tarjeta"	→	`enmascarar_tarjeta`	**0,43**
"interés de la cuenta de ahorros"	→	`calcular_interes_mensual`	**0,39**
"cambiar el correo del cliente"	→	`actualizarDatosContacto`	**0,45**
"transacción sospechosa"	→	`detectar_fraude_por_monto` no aparece	**0,00**

El código es un lenguaje: por eso los LLM programan

1

Gramática estricta

Patrones que se repiten millones de veces en repositorios públicos.

2

Se puede verificar

Compila o no, el test pasa o no: señales claras para entrenar y evaluar.

3

Mucho es repetitivo

CRUD, validaciones, tests, documentación, conversión entre formatos.

Tareas: completar · explicar · traducir · probar · refactorizar · documentar

Lo que no vio: sus reglas de negocio, su arquitectura interna, sus lineamientos de seguridad.

Del modelo base al asistente que usan en VS Code

1

Preentrenamiento

Predice texto y código de enormes corpus

→

2

Ajuste con instrucciones

Aprende a seguir pedidos y a conversar

→

3

Ajuste por preferencias

Se refuerzan respuestas útiles, seguras y honestas

→

4

Asistente (Copilot, Claude)

Suma contexto del IDE o de archivos, herramientas y reglas de la organización

El modelo es el motor. El asistente decide qué contexto le entrega y qué puede hacer con la respuesta.

Las herramientas del curso

GitHub Copilot y Claude Code, cada uno en VS Code y en la terminal

Las herramientas del curso, de un vistazo

VS Code

El editor

Editor de código gratuito de Microsoft. Aquí se abre el proyecto, se escribe, se ejecuta y se conversa con los asistentes. Se amplía con extensiones.

Git

El historial

Registra cada cambio del código. Con agentes es la red de seguridad: ver qué cambió, revisarlo y deshacerlo.

GitHub Copilot

Asistente de IA de GitHub

Sugiere código mientras escriben, responde en un chat y actúa como agente. La organización paga la licencia.

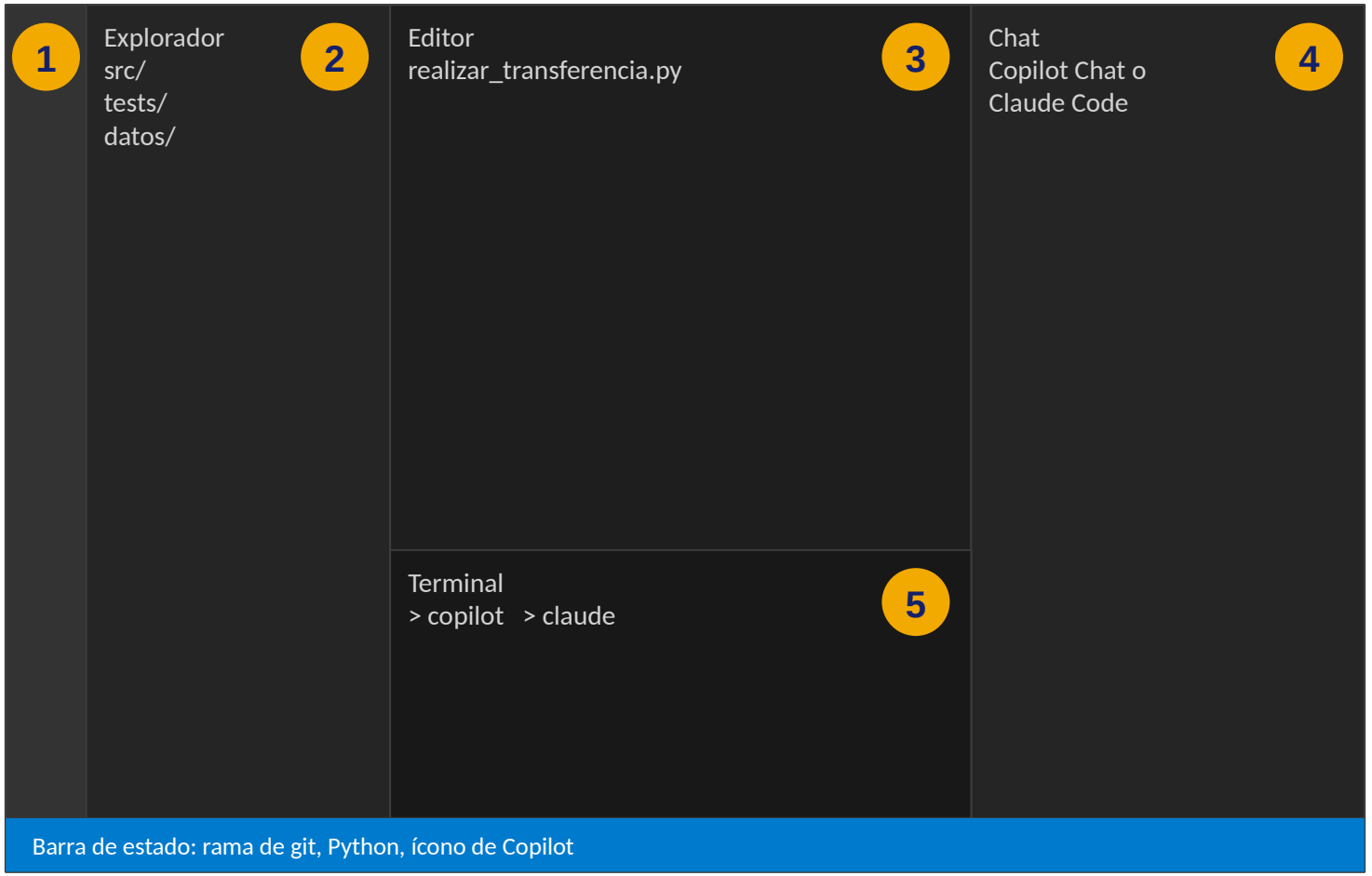
Claude Code

Agente de IA de Anthropic

Agente de programación que lee el proyecto, edita archivos y ejecuta comandos con su permiso.

VS Code y Git son el terreno; Copilot y Claude Code son los asistentes que trabajan en él. El curso enseña a usar los dos asistentes, no a elegir uno.

VS Code: dónde está cada cosa



1 Barra de actividad: explorador, búsqueda, git, extensiones

2 Explorador: los archivos del proyecto

3 Editor: el código; aquí aparecen las sugerencias y Ctrl+I

4 Panel de chat: Copilot Chat o la extensión de Claude Code

5 Terminal integrada: Ctrl+ñ o Ctrl+`; aquí corren copilot y claude

GitHub Copilot: qué es y cómo se usa

Qué es

- Asistente de IA de GitHub, empresa de Microsoft
- Nació en 2021 como autocompletado; hoy también conversa y actúa como agente
- Licencia por persona, pagada por la organización
- Usa modelos de varios proveedores: Anthropic, OpenAI, Google y otros
- Funciona en VS Code, JetBrains, Visual Studio, github.com y la terminal

Sugerencias en línea

Mientras escriben aparece código en gris; Tab lo acepta, Esc lo descarta.

Chat en línea · Ctrl+I

Preguntar o pedir un cambio sobre el código seleccionado, sin salir del editor.

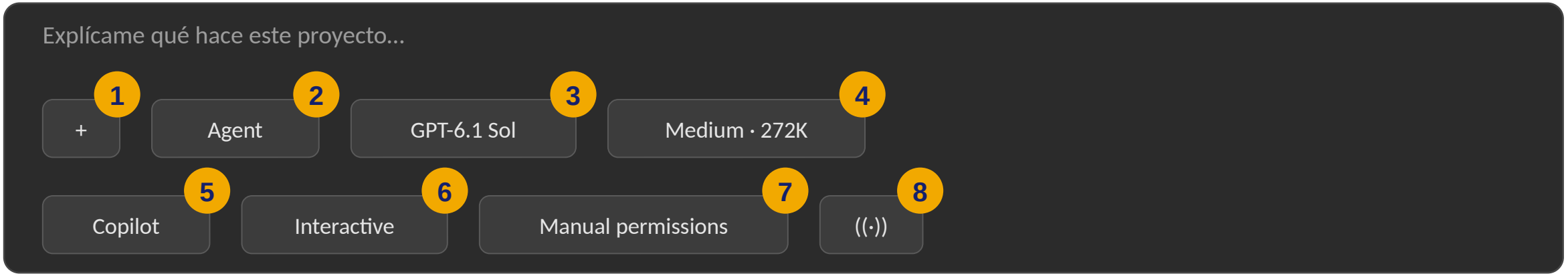
Panel de chat

Conversar sobre el proyecto o pedirle que actúe como agente. Sus controles, en las láminas siguientes.

Copilot CLI

El mismo asistente en la terminal: copilot. Se instala aparte; se ve en la sesión 2.

Copilot Chat por dentro: los controles de la barra



1. **Contexto:** agregar archivos, carpetas o #codebase
2. **Agente:** qué rol cumple: Ask, Agent, Plan o uno personalizado
3. **Modelo:** qué modelo responde
4. **Razonamiento y contexto:** cuánto piensa y cuántos tokens caben
5. **Dónde corre:** Copilot, Local o Cloud
6. **Modo de trabajo:** Interactive, Plan o Autopilot
7. **Permisos:** qué puede hacer sin preguntar
8. **Acceso remoto:** usar esta sesión desde otro equipo

2, 6 y 7 son tres preguntas distintas: qué rol cumple, cuánta autonomía tiene y qué puede hacer sin pedir permiso.

Agente: qué rol cumple Copilot

Agente	Lee el proyecto	Edita archivos	Ejecuta comandos	Úselo para...
Ask	Sí	No	No	Entender código, preguntar, explorar ideas sin riesgo
Plan · /plan	Sí	No	No	Investigar una tarea y proponer un plan para revisar
Agent	Sí	**Sí, con revisión**	**Sí, según permisos**	Implementar, corregir, correr pruebas: tareas de varios pasos
Personalizados	Según su definición	Según su definición	Según su definición	Tareas repetidas del equipo: revisor, generador de pruebas...

Agentes personalizados: los que aparecen debajo de Ask y Agent. Los agregan extensiones o el equipo, con archivos .agent.md. Cada máquina ve una lista distinta.

Regla práctica: empiecen en Ask para entender, pidan un plan si la tarea es grande y pasen a Agent cuando sepan qué quieren cambiar.

Dónde corre, cuánta autonomía y qué permisos

5 · Dónde corre

Copilot: recomendado. Corre aparte del editor, sigue si se cierra la ventana y se comporta igual que Copilot CLI

Local: dentro de VS Code; se detiene al cerrar la ventana

Cloud: en la nube sobre un repositorio de GitHub; entrega un pull request

6 · Modo de trabajo

Interactive: colaboran paso a paso

Plan: primero el plan; ejecuta cuando usted lo aprueba

Autopilot: trabaja solo: aprueba herramientas, reintenta y decide; consume más créditos

7 · Permisos

Manual: pregunta antes de lo que no está preaprobado

Assisted: experimental; otro modelo evalúa el riesgo de cada acción

Allow all: ejecuta todo sin preguntar

Sandbox de terminal: limita archivos y red de los comandos

Para el curso y para código de la organización: Copilot · Interactive · Manual permissions · acceso remoto apagado. Nada de Autopilot ni Allow all.

Claude Code: qué es y cómo se usa

Qué es

- Agente de programación de Anthropic, la empresa que crea los modelos Claude
- Nació en la terminal (2025): se le pide una tarea y la ejecuta por pasos
- Usa solo modelos Claude
- Acceso con el plan de Claude de la organización o por API o nube
- Funciona en la terminal, en VS Code y JetBrains, en una app de escritorio y en la web

Claude Code CLI

claude en la terminal: lee el proyecto, edita archivos y ejecuta comandos, pidiendo permiso.

Extensión de VS Code

Panel propio (ícono de chispa) con el mismo agente: ve el archivo abierto y muestra los cambios como diff.

Memoria del proyecto

Lee CLAUDE.md al empezar: las reglas del equipo. /init crea uno a partir del código.

Modo plan y permisos

Planea sin tocar nada antes de actuar; cada edición o comando se puede aprobar o rechazar.

Palabras que van a escuchar en el curso

Prompt	Lo que se le escribe al asistente: la pregunta o la tarea.
Contexto	Todo lo que el modelo ve al responder: el prompt, archivos, instrucciones e historial.
Agente	Asistente que, además de responder, lee, edita y ejecuta en ciclo hasta terminar.
Modo plan	El agente propone los pasos y espera su aprobación antes de tocar archivos.
Extensión	Complemento que se instala en VS Code; Copilot Chat y Claude Code lo son.
CLI	Interfaz de línea de comandos: el asistente usado desde la terminal.
Instrucciones	Archivos con las reglas del equipo que el asistente lee solo: AGENTS.md, CLAUDE.md.
MCP	Protocolo para conectar el asistente con otras herramientas y datos. Sesión 8.

Cómo trabaja un agente de código

1

Lee el contexto

Su pedido, los archivos abiertos o del proyecto y las instrucciones del equipo

→

2

Planea

Decide qué pasos seguir; en modo plan se los muestra antes de tocar nada

→

3

Actúa

Lee y edita archivos, ejecuta comandos y pruebas, siempre con su permiso

→

4

Verifica

Lee el resultado o el error y decide el siguiente paso

Repite el ciclo hasta terminar. Por eso un agente puede hacer tareas de varios pasos, y por eso importan dos cosas: el contexto que le damos y los permisos que le aprobamos.

En Claude Code 101 este flujo se enseña como: explorar → planear → codificar → commit.

Dos asistentes, cada uno en VS Code y en la terminal

	En VS Code Panel dentro del editor	En la terminal CLI, también en la terminal de VS Code
GitHub Copilot	**Copilot Chat** Modos Ask y Agent, chat en línea (Ctrl+I) y autocompletado. Ve el archivo abierto y lo que le adjunten.	**Copilot CLI** copilot en la consola: lee, edita y ejecuta con su permiso. Modo plan con Shift+Tab. Misma licencia.
Claude Code	**Extensión de VS Code** Panel propio (ícono de chispa). Ve el archivo abierto, propone cambios como diff y tiene modo plan (/plan).	**Claude Code CLI** claude en la consola: el mismo agente, con todos sus comandos. /init crea un CLAUDE.md del proyecto.

Mismas ideas, distintos nombres

	Copilot Chat	Copilot CLI	Claude Code · VS Code	Claude Code CLI
Dónde	Panel en VS Code	Terminal	Panel en VS Code	Terminal
Contexto	Archivo abierto y adjuntos	La carpeta donde lo abren	Archivo abierto y @archivo	La carpeta donde lo abren
Instrucciones del equipo	AGENTS.md y .github/copilot-instructions.md	AGENTS.md y .github/copilot-instructions.md	CLAUDE.md (puede importar AGENTS.md)	CLAUDE.md (puede importar AGENTS.md)
Modo plan	Pedirle que planee primero	Shift+Tab	/plan	Shift+Tab
Para empezar	Escribir en el chat	copilot · /login · /model	Ícono de chispa	claude · /init · /status

Fíjense en la fila de instrucciones: un mismo AGENTS.md puede guiar a las cuatro. Esa es la base del curso.

Qué modelos hay detrás de cada asistente

GitHub Copilot

Varios proveedores · selector de modelo y modo Auto

Anthropic: Claude Opus 5.5, Sonnet 5.5, Haiku 4.5

OpenAI: GPT-6.1 Sol, GPT-6 Luna, GPT-5.6, GPT-5.3-Codex

Google y Microsoft: Gemini Flash, MAI-Code

Auto: Copilot elige el modelo por ustedes

Qué ven depende del plan y de la política de TI

Claude Code

Solo modelos Claude · /model y /effort

Haiku 5.5: el más rápido y barato

Sonnet 5.5: el equilibrado, para el día a día

Opus 5.5: el predeterminado; tareas complejas

Fable 5.1: lo más difícil y largo, si está habilitado

opusplan: Opus para planear y Sonnet para ejecutar

Claude dentro de Copilot no es lo mismo que Claude Code: el modelo puede ser el mismo, pero el agente, el contexto y las herramientas cambian.

¿Qué modelo uso para qué?

Si la tarea es…	Tipo de modelo	En Copilot	En Claude Code
Autocompletar, renombrar, una pregunta corta	**Rápido**	Auto, GPT-6 Luna o un Flash	haiku · /effort low
Escribir una función, sus pruebas, explicar un módulo	**Equilibrado**	Claude Sonnet 5.5 o GPT-6.1 Sol	sonnet · /effort medium
Un bug difícil, una decisión de arquitectura, un refactor grande	**De razonamiento**	Claude Opus 5.5	opus u opusplan · /effort high
Una tarea larga: migrar o analizar un sistema heredado completo	**El más capaz**	Claude Opus 5.5	fable, si la organización lo habilita
Revisar la seguridad de un cambio	**Uno distinto del que lo escribió**	Cambiar de modelo para revisar	Otro modelo o un subagente revisor

Empiecen con el equilibrado. Suban de modelo si falla; bajen si la tarea es repetitiva.

Antes de cambiar de modelo, ajusten el esfuerzo y revisen el contexto: casi siempre el problema es lo que el agente no sabe.

¿Qué modelos tengo yo? Depende de la licencia

Forma de uso	Dónde se ve	Qué lo decide
Copilot Chat	Clic en el nombre del modelo, en la barra del chat	El plan de Copilot (Free, Pro, Pro+, Business, Enterprise) y la política del administrador
Copilot CLI	/model (y /model auto); abajo, los créditos usados	La misma licencia y política que Copilot Chat
Claude Code · extensión	Clic en el modelo (Opus 5.5 · Medium) o /model	El plan de Claude de la organización o el acceso por API o nube
Claude Code CLI	/model para la lista; /status para cuenta y método de acceso	El mismo plan que la extensión

No todos verán la misma lista. Puede que yo no tenga Claude Opus 5.5 en Copilot y ustedes sí, o al revés. Si un modelo no aparece, el plan no lo incluye o la organización no lo habilitó. En la sesión 2 cada uno revisa los suyos.

Recorrido en vivo: un proyecto, cuatro formas de uso

Repositorio de práctica: proyectos/transferencias · servicio de transferencias entre cuentas, con historial git (datos sintéticos)

```
Explícame qué hace este proyecto, cómo está organizado y cómo se prueba.
```

Copilot Chat	Claude Code · VS Code	Copilot CLI	Claude Code CLI
Ask → «¿Qué reglas de negocio aplica una transferencia?» · Ctrl+I sobre calcular_gmf · Agent para correr pruebas	Con realizar_transferencia.py abierto → «¿Qué pasa si el origen está bloqueado?» · modo de permisos	copilot → confiar en la carpeta → «Ejecuta las pruebas y dime qué cubren» · pide permiso	claude → confiar en la carpeta → «Lee el historial de git: ¿qué quedó pendiente?»

En las cuatro: ¿qué modelo usa y cuáles más ofrece? **Para el grupo: ¿qué leyó cada una? ¿Cuál pidió permiso y para qué?**

Hacia dónde vamos: el cerebro del equipo

Cerebro del equipo

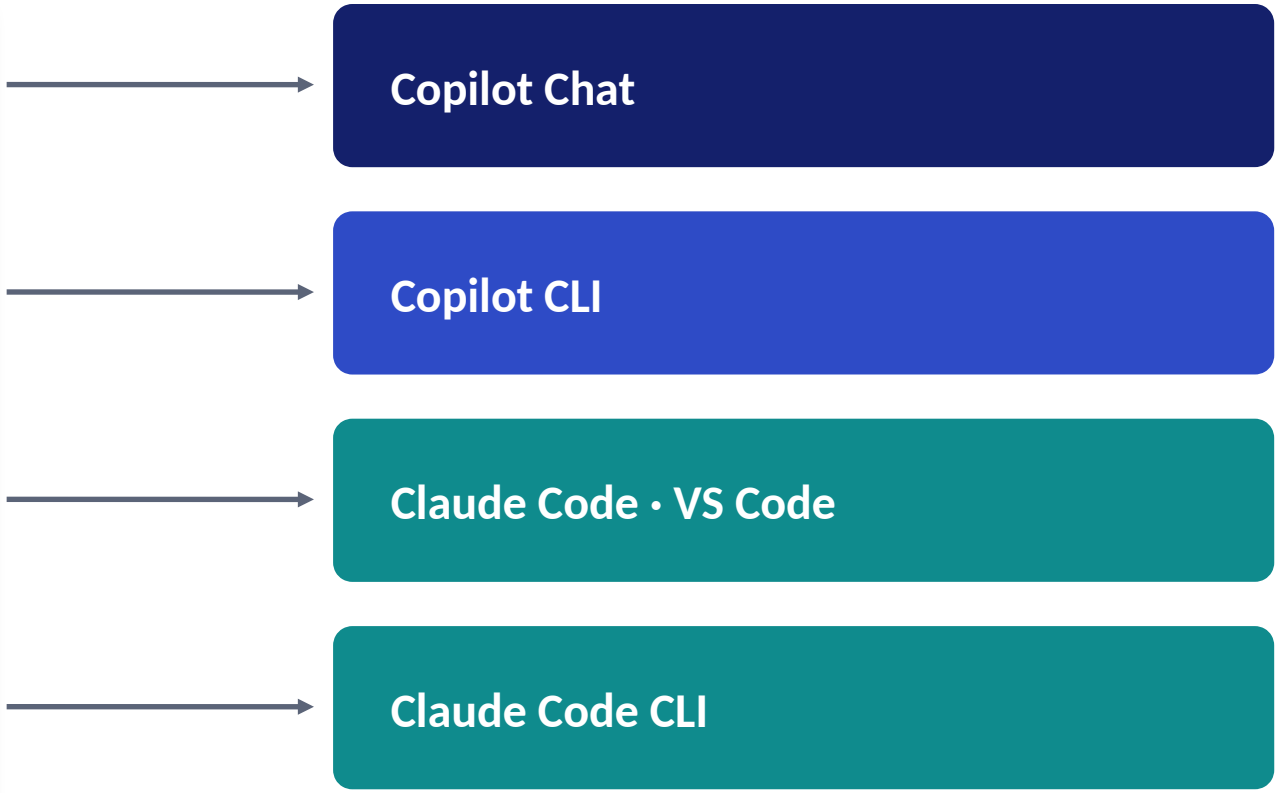
Instrucciones: arquitectura, nombres, "nunca hagas X"

Reglas: distintas para cada tipo de código

Skills: flujos como "nuevo caso de uso"

Agentes: revisor de código y de seguridad

Hooks: pruebas y lint en cada cambio



Los estándares del equipo, escritos una vez y aplicados igual en el editor y en la terminal. Lo construimos por partes, de la sesión 3 a la 10.

Manos a la obra

Un requisito de negocio, dos asistentes, una aplicación

Demo A: dos límites, en las dos herramientas

1 Alucinación

```
¿Cómo uso
pandas.DataFrame.to_bank_ledger() para
exportar un libro contable?
```

Esa función no existe. Misma pregunta en Copilot Chat y en Claude Code: ¿alguno la inventa?

2 Datos sensibles

```
API_KEY = "sk-demo-…"
Archivo abierto en VS Code con los dos
paneles
```

Los dos ven el archivo abierto, aunque no se lo adjunten. ¿Qué acaba de salir del equipo?

El reto: un simulador de crédito de consumo

Pregunta de negocio

Un cliente quiere un crédito de libre inversión. ¿Cuánto pagará cada mes, cuánto le costará en total y le alcanza con sus ingresos?

Hoy cada oficina lo calcula en una hoja de cálculo distinta y los resultados no coinciden. Queremos un simulador único, con las reglas del banco en un solo lugar, para que el asesor lo use frente al cliente.

`REQUISITOS.md`

- Datos: monto, plazo, tasa E.A., ingresos y otras cuotas
- Validaciones: montos, plazos y tasa máxima
- Cuota fija, tabla de amortización y resumen
- Capacidad de pago: endeudamiento hasta 40 %
- Una app web con Streamlit: formulario, semáforo, tabla y gráfico

`AGENTS.md · reglas del equipo`

- Dinero en Decimal, redondeo al peso; tasas E.A.
- Dominio sin interfaz; nombres en español; una prueba por regla

Mismos requisitos, dos asistentes, en paralelo

1

Leer

REQUISITOS.md y AGENTS.md: qué construir y cómo lo hace el equipo

2

Planear

Copilot con /plan; Claude Code en modo plan. Revisamos el plan antes de aprobar

3

Construir

Cada agente crea el dominio, la app y las pruebas, pidiendo permiso para ejecutar

4

Verificar

Sus pruebas y, además, las pruebas de aceptación que el asistente no vio

5

Comparar

Abrimos las dos apps con el mismo ejemplo: ¿coinciden las cifras?

resultados/simulador-copilot
Copilot Chat · Agent · lee AGENTS.md

resultados/simulador-claude-code
Claude Code CLI · lee CLAUDE.md → AGENTS.md

Mientras trabajan: ¿qué archivos leyó cada uno? ¿Qué decidió que no estaba en los requisitos? ¿Cuándo pidió permiso?

Lo que acaban de ver es el curso

Lo que pasó en la demo	Dónde lo aprendemos a hacer bien
Unos requisitos claros y un contrato de nombres	**S3 prompt y context engineering · S4 de historia de usuario a caso de uso**
Las reglas del equipo en AGENTS.md, leídas por los dos asistentes	**S3 instrucciones del equipo · S8 empaquetar el cerebro del equipo**
Un plan revisado antes de tocar código; permisos para ejecutar	**S2 Copilot y Claude Code a fondo: modos, planes y permisos**
Pruebas propias y pruebas de aceptación que el agente no vio	**S6 pruebas y refactorización con agentes**
Revisar lo que generó: errores, seguridad, datos sensibles	**S5 depuración y code review · S7 seguridad**
Una estructura de proyecto repetible para cada caso nuevo	**S4 la plantilla del equipo · S9–S10 su propio reto**

Pregunta para el grupo: ¿qué caso de su trabajo les gustaría construir así en el reto final?

En qué es bueno y en qué falla

Funciona bien

- Código repetitivo y plantillas
- Explicar y resumir código heredado
- Primer borrador de pruebas y documentación
- Traducir entre lenguajes y formatos
- Patrones comunes y librerías populares

Falla o hay que verificar

- Inventa funciones, APIs y paquetes
- No conoce las reglas internas de la organización (tasas, redondeos, límites)
- Lógica de negocio sutil y casos borde
- Conocimiento con fecha de corte
- No determinista; el tono no indica certeza

Riesgos que importan en el sector financiero

Fuga de datos

Datos de clientes, secretos o código propietario en un prompt

Código inseguro

Inyección SQL, validaciones débiles, datos personales en logs

Alucinaciones

APIs o reglas de negocio inventadas que pasan la revisión

Paquetes falsos

Dependencias inexistentes que un atacante puede registrar

Inyección de prompts

Instrucciones ocultas en código, tickets o documentos

Propiedad intelectual

Código sugerido con licencias incompatibles

Cinco reglas para usar IA con código de la organización

1	**Ustedes son los autores**	Lo que aceptan es su código: lo entienden, lo revisan y responden por él.
2	**Mínimo dato**	Nunca datos reales de clientes ni secretos en un prompt. Datos sintéticos o anonimizados.
3	**Verificar con pruebas**	Una sugerencia no está lista hasta que pasa pruebas y revisión.
4	**Solo herramientas autorizadas**	Copilot y Claude Code, con las cuentas y políticas de la organización.
5	**Dejar trazabilidad**	Registrar qué se generó con IA en commits y pull requests según los lineamientos internos.

Para profundizar: dos rutas oficiales

Microsoft Learn

GH-600 · Developing in Agentic AI Systems

Hoy y S2: Foundations of Agentic AI in GitHub

S3–S4: Agent Architecture and SDLC Integration

S5–S8: MCP · Multi-agent · Evaluation · Governance

Al final: GitHub Certified: Agentic AI Developer

Anthropic Academy

Claude Code 101

Hoy: What is Claude Code? · How Claude Code works

Sesión 2: Daily workflows · Explore → plan → code → commit

Sesión 3: Context management · The CLAUDE.md file

Después: Subagents · Skills · MCP · Hooks

El mapa completo por sesión está en REFERENCIAS.md, en la carpeta del curso. Son opcionales y no reemplazan los laboratorios.

Para la sesión 2

1

Preparar su máquina

Guía preparacion/README.md: VS Code, Copilot CLI y Claude Code. Envíen la captura del verificador.

2

Cazar errores

autonomo/ validador_transacciones.py con Copilot y su BITACORA.md. Hay al menos 5 problemas.

3

Lecturas y una idea

Primer módulo de GH-600 (Microsoft Learn), las primeras lecciones de Claude Code 101 (Anthropic Academy) y una tarea real para el reto.

Un LLM predice tokens; su trabajo es darle buen contexto y verificar lo que produce.